

LAB 1: Requirements Engineering and UML Use-Case Modelling

1. Problem Context & Overview

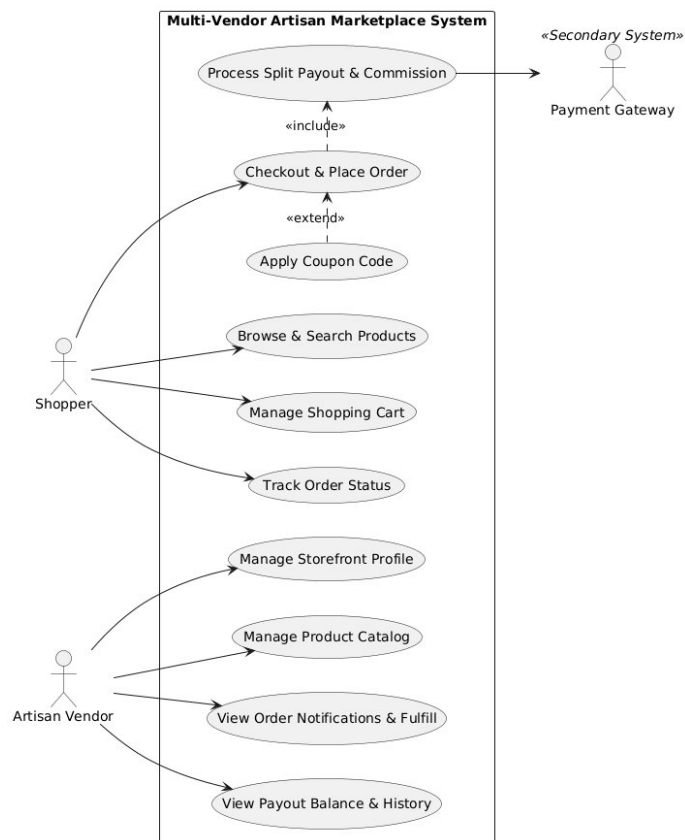
An online marketplace enabling independent craftspeople to set up storefronts, manage product catalogs, receive orders, and receive automated split payouts with platform commission deductions.

Target Stakeholders / Actors: Shopper, Artisan Vendor

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR - 001	Functional	The system should split customer cart payments at checkout, allocating respective earnings to each of the multiple vendors, after deducting 5% platform fee.	High	Pass: Split payout calculations exactly balance total cart value minus fees	Ensures accurate multi-vendor revenue distribution and automated platform fee collection
FR - 002	Functional	System should allow artisan vendors to create, update and manage product catalog listings including titles, descriptions, stock counts and prices.	High	Pass: Product edits immediately reflect on the public storefront.	Enables artisans to maintain accurate inventory and product details independently.
FR - 003	Functional	System should allow shoppers to add items from multiple distinct artisan storefronts into a unified shopping cart and checkout.	High	Pass: Shopper successfully completes a single order containing items from 2 different vendors.	Minimizes checkout friction for buyers purchasing from diverse craftspeople.
FR - 004	Functional	System should notify artisan immediately via email and dashboard alerts upon receipt of a confirmed order of their products.	Medium	Pass: Order notifications arrive within 10 seconds of successful checkout.	Guarantees timely order fulfillment and packaging by independent artisans.

FR - 005	Functional	System should allow artisans to configure their digital storefront profiles.	Medium	Pass: Profile changes save and render correctly across shopper-facing storefront pages	Helps craftspeople establish their individual brand identity within the marketplace.
NFR - 001	Non - Functional	Product catalog must high resolution image image rendering with CDN caching with load times < 500ms.	High	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load	High-resolution imagery is critical for showcasing artisan craftsmanship without degrading browsing speed.
NFR - 002	Non - Functional	System must encrypt all payment processing and banking information in transit (TLS) and at rest (AES - 256).	High	Pass: Automated vulnerability and penetration tests verify zero plaintext exposure of financial credentials.	Protects sensitive financial credentials during automated multi-vendor payouts and customer checkouts.

UML Diagram



Use Case ID: UC-001

Use Case Name: Checkout and Split Payment

Primary Actor: Shopper

Secondary Actors: Payment Gateway, Artisan Vendor

1. Brief Description

Enables a shopper to review their unified cart (containing items from multiple artisan vendors), enter payment details, and complete the order. The platform deducts a 5% commission and splits the remaining balances to the respective vendors.

2. Preconditions

- Shopper is authenticated and logged into their marketplace account.
- Shopping cart contains one or more product listings from artisan vendor(s) with valid quantities.
- External payment gateway services and merchant settlement APIs are reachable.

3. Postconditions

Success:

- Order status is marked as Confirmed in the database.
- Customer payment is successfully authorized and captured.
- Platform deducts 5% from item subtotals as marketplace commission.
- Net earnings (95%) are allocated to the respective artisan vendor payout balances.
- Product stock levels are decremented across all purchased catalog listings.
- Automated order notifications are dispatched to buyer and artisans.

Failure:

- Transaction is aborted.
- No funds are captured.
- Inventory remains unchanged.
- Error state is displayed to the shopper.

4. Main Success Scenario (MSS)

1. Shopper navigates to the shopping cart page and clicks Proceed to Checkout.
2. System validates real-time stock availability and pricing for all items across vendors.
3. Shopper confirms delivery address and selects a payment method (e.g., Credit Card/UPI/Net Banking).
4. System calculates the Total Order Value:
$$\text{Total Cart Value} = \sum(\text{Item Price} \times \text{Quantity}) + \text{Shipping Fee} + \text{Taxes}$$
5. Shopper submits the payment form.
6. System routes the total authorization request to the Payment Gateway.
7. Payment Gateway verifies credentials and returns a Transaction Successful token with the captured amount.
8. System executes the multi-vendor split calculation:
 - Platform Cut: $\text{Fee} = 0.05 \times \text{Vendor Item Subtotal}$
 - Vendor Payout: $\text{Net Payout} = \text{Vendor Item Subtotal} - \text{Fee}$
9. System credits each artisan vendor's payout ledger with their respective net payout balance.
10. System decrements product stock counts in the catalog database.
11. System generates a unique Order ID and digital receipt.

12. System sends confirmation alerts to the shopper and fulfillment notifications to artisans.

13. Shopper is redirected to the Order Confirmation page with an itemized receipt.

5. Alternate Flows

AF-1: Payment Gateway Authorization Failure

1. At Step 6, the Payment Gateway declines the transaction (e.g., insufficient funds, network timeout, or invalid authentication).

2. System logs the failed attempt and keeps the cart intact.

3. System rolls back any temporary stock holds.

4. System displays: "Payment could not be processed. Please verify your card details or try a different payment method."

5. Shopper is prompted to retry payment or select an alternative method without losing cart items.

AF-2: Mid-Checkout Inventory Depletion

1. At Step 2, one or more items become out of stock prior to payment submission.

2. System alerts the shopper, updates cart quantities to available limits, and recalculates the total.

3. Shopper confirms the updated cart to resume from Step 3.